Project - Low Level Design

on Chat with

Logistics Document (RAG)

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	REET SOOD	EN22CS301796
2.	PARTH AKHAND	EN22CS301687
3.	ROSHNI RAWAT	EN22CS301826
4.	PRAKRATI RASANIYA	EN22CS301722
5.	RIYANSHI JAIN	EN22CS301816
6.	RISHI YADAV	EN22CS301806

Group Name: Group 02D7

Project Number: GAI-25

Industry Mentor Name: Aashruti Shah Ma'am

University Mentor Name: Dr. Hemlata Patel Ma'am

Academic Year: 2022-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

The project **"Chat with Logistics Document (RAG)"** implements a Retrieval-Augmented Generation (RAG) system for logistics document intelligence. The system enables users to upload professional logistics documents such as reports, manuals, and bills of lading, and interact with them using a conversational interface.

The application processes documents through a pipeline of text extraction, chunking, and vector embedding. It uses a vector database for semantic retrieval and integrates a Large Language Model (LLM) to generate accurate, context-aware responses strictly based on the uploaded documents.

1.1 Scope of the Document

This document provides a detailed low-level design of the system, including:

- Internal architecture and modules
- Data processing pipeline
- Component interactions
- API-level functionality

The scope is limited to application-level design and excludes deployment infrastructure and scaling strategies.

1.2 Intended Audience

This document is intended for:

- Developers and maintainers of the system
- System architects and reviewers
- Academic evaluators
- Students studying RAG-based applications

1.3 System Overview

The system allows users to:

- Upload logistics-related PDF documents
- Process and store document content as vector embeddings
- Ask natural language questions
- Receive responses grounded strictly in the document content

Technologies Used:

- Python
- Streamlit (User Interface)
- ChromaDB (Vector Database)
- HuggingFace Embeddings
- Groq LLM (LLaMA 3.1 8B)
- LangChain Framework

2. System Design

The system follows a **modular RAG architecture** consisting of the following layers:

1. **Presentation Layer (UI)** – Streamlit-based interface
2. **Processing Layer** – Document ingestion and chunking
3. **Storage Layer** – Vector database (ChromaDB)
4. **Retrieval Layer** – Semantic similarity search
5. **Generation Layer** – LLM-based response generation

Each layer operates independently and communicates through well-defined functions.

2.1 Application Design

Frontend:

- Built using Streamlit
- Provides:
  - File upload interface
  - Chat-based interaction
  - Real-time response display

Backend:

- Handles:
  - PDF loading and processing
  - Text chunking
  - Embedding generation
  - Vector storage and retrieval
  - LLM-based response generation

2.2 Process Flow

1. User uploads PDF documents via UI
2. Files are stored in the `data/` directory
3. System extracts text using PDF loader
4. Text is split into chunks with overlap
5. Each chunk is converted into embeddings
6. Embeddings are stored in ChromaDB
7. User submits a query
8. System retrieves top relevant chunks using similarity search
9. Retrieved context is combined with the query
10. Prompt is sent to the LLM
11. LLM generates a grounded response
12. Response is displayed in the chat interface

2.3 Information Flow

```
User → Streamlit UI → File Upload
        ↓
Document Processor → Text Extraction → Chunking
        ↓
Embedding Model → Vector Embeddings
        ↓
ChromaDB → Storage
        ↓
User Query → Retriever → Relevant Chunks
        ↓
RAG Pipeline → LLM (Groq) → Response
        ↓
Streamlit UI → Display Output
```

2.4 Components Design

1. `app.py`

- Main application entry point
- Handles:
  - File uploads
  - Chat interface
  - Session management
  - Calls backend functions

2. `document_processor.py`

- Loads PDF documents from `data/` directory
- Splits text using `RecursiveCharacterTextSplitter`
- Uses:
  - Chunk size: 1000
  - Overlap: 200

3. `vector_store.py`

- Generates embeddings using HuggingFace model (`all-MiniLM-L6-v2`)
- Stores embeddings in ChromaDB (`chroma_db/`)
- Provides:
  - `build_vector_store()`
  - `get_vector_store()`

4. `retriever.py`

- Performs semantic similarity search
- Retrieves top-k relevant chunks (default k=3)
- Returns document chunks with similarity scores

5. `rag_pipeline.py`

- Integrates retrieval and generation
- Uses Groq LLM (`llama-3.1-8b-instant`)
- Applies strict prompt to:
    - Restrict answers to context
    - Prevent hallucination
- Generates final response

2.5 Key Design Considerations

1. **Accuracy**
- Uses semantic similarity search for relevant retrieval
2. **Grounded Responses**
- Strict prompt ensures no external knowledge is used
3. **Performance**
- ChromaDB enables fast retrieval of embeddings
4. **Cost Efficiency**
- Local embedding model reduces API dependency
5. **Modularity**
- Independent components for easy maintenance
6. **Scalability**
- Can extend to larger datasets and cloud databases
7. **Usability**
- Simple and intuitive UI using Streamlit

2.6 API Catalogue

1. `build_vector_store()`

- Description: Processes documents and builds vector DB
- Input: None
- Output: ChromaDB instance

2. `get_vector_store()`

- Description: Loads existing vector DB
- Input: None
- Output: Database object

3. `retrieve_context(query, k=3)`

- Description: Retrieves relevant document chunks
- Input: User query
- Output: List of chunks with similarity scores

4. `generate_answer(query)`

- Description: Generates final response using LLM
- Input: User query
- Output: Context-based answer

3. References

- LangChain Documentation
- ChromaDB Documentation
- HuggingFace Transformers
- Groq API Documentation
- Retrieval-Augmented Generation (RAG) Concepts